

# LeetCode 1109: Corporate Flight Bookings

Goldman Sachs

© Authored by Arpit Raj, LNMIIT Jaipur | July 11, 2026

## Approach 1: Brute Force

**Time Complexity:**  $O(n \times m)$  where  $n$  is the number of flights and  $m$  is the number of bookings.

This naive approach iterates through each booking and increments the seat count for every flight in the given range one by one.

```
class Solution {
public:
    vector<int> corpFlightBookings(vector<vector<int>>& bookings, int n) {
        vector<int> ans(n, 0); // initially koi seat book nhi he

        for(auto &book:bookings){
            int from = book[0] - 1; // 0 based indexing
            int to = book[1] - 1;
            int num = book[2];

            for(int i = from; i <= to; i++){
                ans[i] += num;
            }
        }

        return ans;
    }
};
```

## Approach 2: Difference Array (Optimal)

**Time Complexity:**  $O(n + m)$

Uses the **Difference Array** technique to process range updates in  $O(1)$  time. After all ranges are updated via endpoints ( $\text{diff}[\text{from}] += \text{num}$  and  $\text{diff}[\text{to}+1] -= \text{num}$ ), computing the prefix sum of the array yields the final continuous booking counts across all flights.

```
class Solution {
public:
    vector<int> corpFlightBookings(vector<vector<int>>& bookings, int n) {
        vector<int> diff(n, 0);

        for(auto &book:bookings){
            int from = book[0] - 1; // 0 based indexing
            int to = book[1] - 1;
            int num = book[2];

            diff[from] += num;

            if(to + 1 < n)
                diff[to + 1] -= num;
        }

        vector<int> pref(n, 0);
        pref[0] = diff[0];
        for(int i = 1; i < n; i++){
            pref[i] = pref[i - 1] + diff[i];
        }

        return pref;
    }
};
```